
Enregistrement

Enregistrement

Les **enregistrements** sont des classes dont les instances ont pour unique responsabilité d'**agréger plusieurs informations** en une seule valeur. Par exemple, pour représenter :

- les données biométriques d'un individu (nom, âge, poids, ...),
- ou les coordonnées d'une grille (ligne et colonne),
- ou les métadonnées d'un fichier de musique,...

Sans enregistrement

Sans enregistrement, on peut utiliser une classe :

```
public final class BiometricData {  
    private final String name;  
    private final int height; // in cm  
    private final double weight; // in kg  
    public BiometricData(String name,  
                           int height,  
                           double weight) {  
        this.name = name;  
        this.height = height;  
        this.weight = weight;  
    }  
    public String name() { return name; }  
    public int height() { return height; }  
    public double weight() { return weight; }  
}
```

Avec enregistrement

Les enregistrements simplifient l'écriture de telles classes au minimum :

```
record BiometricData(String name,  
                      int height,  
                      double weight) {}
```

Cette ligne est équivalente à toute la classe précédente ! (et même un peu plus : equals, toString, hashCode)

Syntaxe générale :

```
record NomDeClasse(descripteur) { ... }
```

Définition d'un enregistrement

Descripteur d'enregistrement : liste des paramètres (type + identifiant) déclarées à la définition de l'enregistrement.

Les enregistrements sont des classes avec :

- pour chaque paramètre du descripteur :
 - une **propriété privée finale** de type et d'identifiant identiques au paramètre ;
 - un **accesseur public** ayant le même identifiant ;
- un constructeur public, le **constructeur canonique**, ayant les mêmes paramètres que le descripteur,
- des **méthodes** hashCode, equals, toString appropriées.

Déclarations supplémentaires

On peut ajouter :

- une **redéfinition du constructeur** ou des accesseurs,
- des **méthodes** arbitraires,
- des **propriétés statiques** et/ou un initialiseur statique (pour initialiser des constantes, cf. cours4.**Point2D**) ;
- des types génériques (l'enregistrement peut être générique) ;
- des déclarations d'implémentation d'interfaces.

Par contre, **on ne peut pas ajouter de propriétés d'instance** (non-statique).

Quand utiliser des enregistrements ?

Vocation d'un objet standard : avoir une responsabilité, rendre un service.

Vocation d'un enregistrement : être une **donnée**.

Critère d'utilisation des enregistrements :

- Si la classe a pour seul rôle d'agréger des données, sans fonctionnalité associée.
- Seulement si la classe n'a pas de raison d'être étendue plus tard.
- Seulement si les objets de la classe sont immuables.
- Les enregistrements sont particulièrement utiles en tant que classes internes, lorsqu'une méthode doit retourner plusieurs informations

Le sélecteur switch

Le sélecteur instructionnel sur les entiers

Structure de contrôle `switch`

La structure de contrôle `switch` permet de choisir entre plusieurs alternatives, selon une valeur entière :

```
switch (value) {  
    case 1 : return "one";  
    case 2 : return "two";  
    case 3 : return "three";  
    case 4 : return "four";  
    default : return "many";  
}
```

Règles du switch

```
switch (expression entière) {  
    ...  
    case 42 :  
        ...  
    default :  
        ...  
}
```

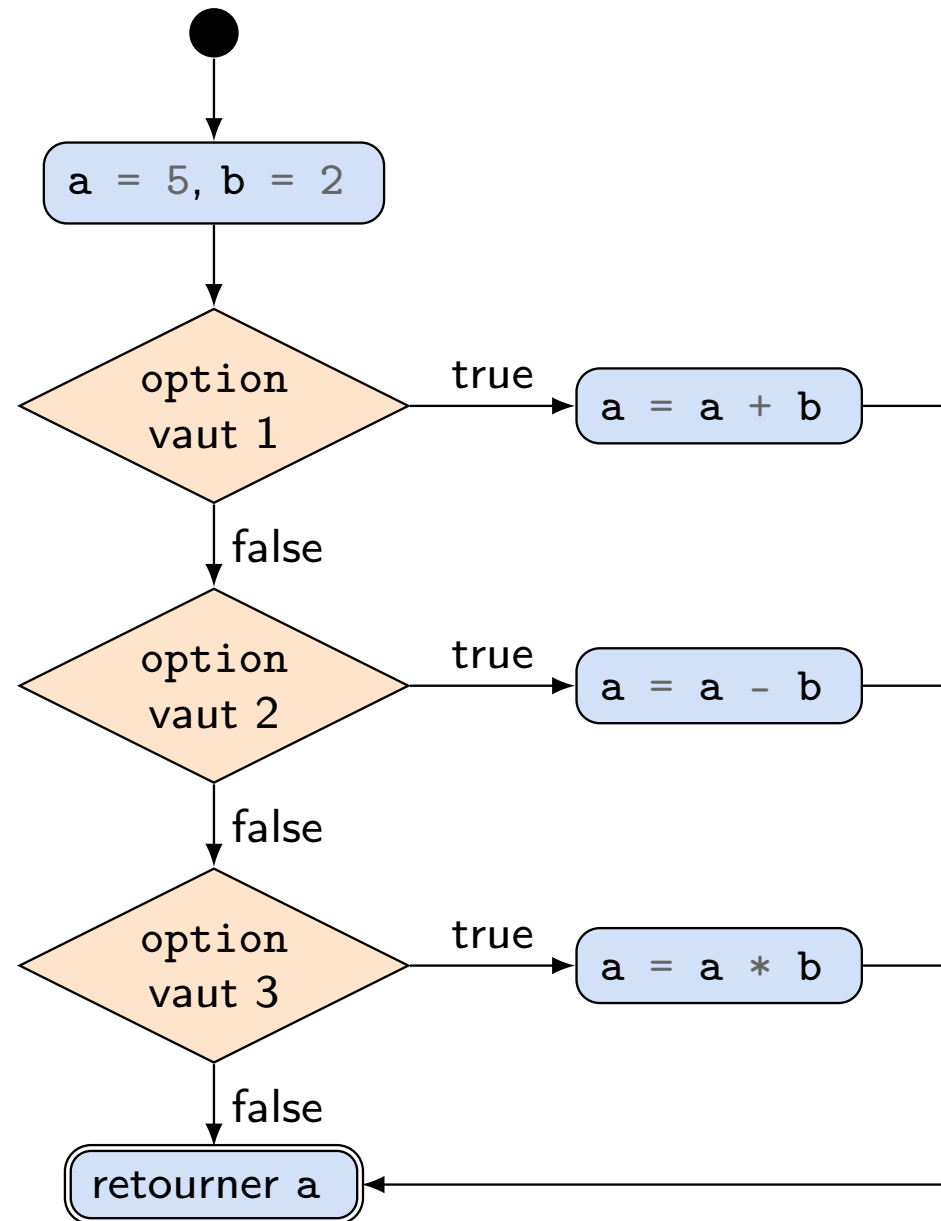
- `case` suivi d'un `littéral` obligatoirement (un entier écrit en chiffre, comme 42).
- L'expression sélecteur est évaluée, puis on saute à l'instruction suivant le `case` correspondant à sa valeur,
- `break` ou `return` quitte le bloc du `switch`.
- Si aucun `case` ne correspond à la valeur évaluée, saute à `default` s'il existe, sinon hors du bloc.

Exemple switch

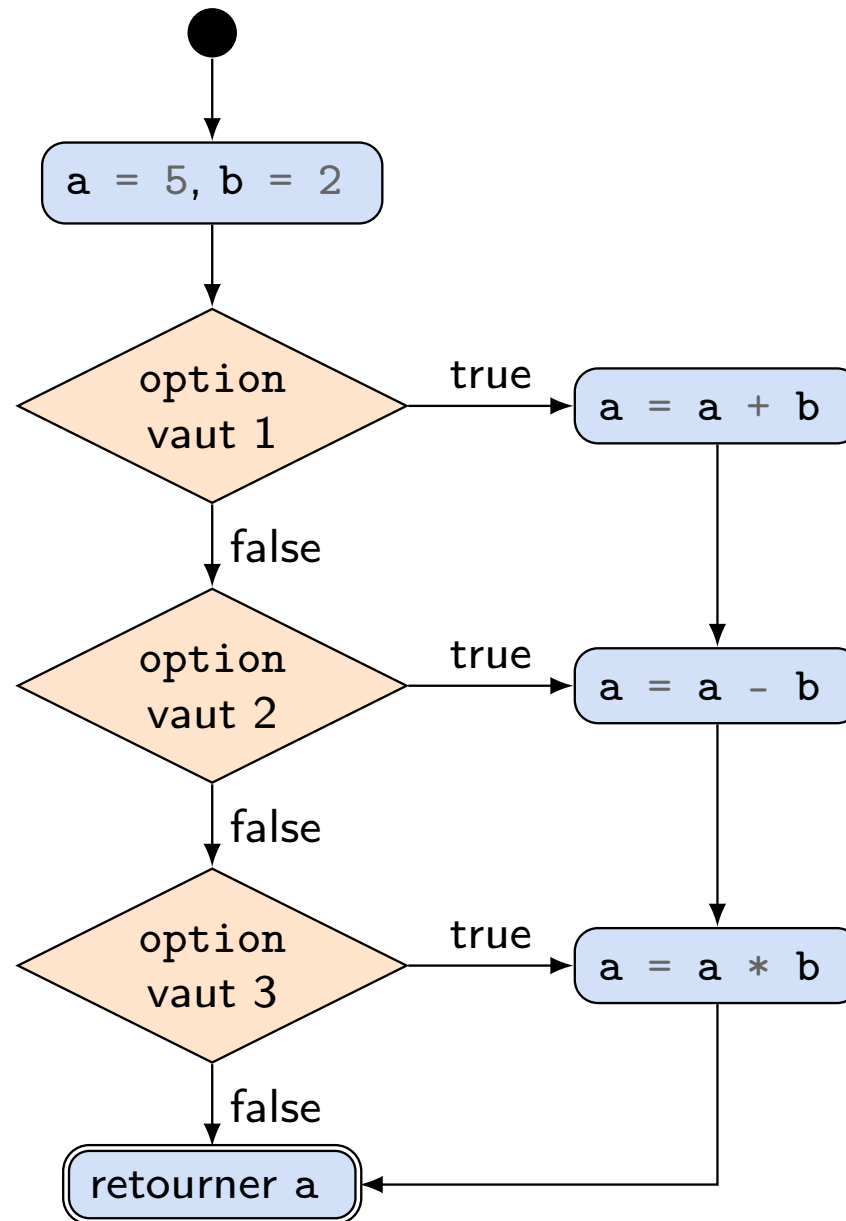
```
public int chooseOperation(int option) {  
    int a = 5;  
    int b = 2;  
    switch (option) {  
        case 1 :  
            a = a + b;  
            break;  
        case 2 :  
            a = a - b;  
            break;  
        case 3 :  
            a = a * b;  
            break;  
    }  
    return a;  
}
```

option	1	2	3	4
résultat	7	3	10	5

Organigramme du switch



Le même programme sans les break



Extension aux enums, String

`switch` peut aussi décider selon une valeur d'un type énumération ou du type `String`.

```
public enum Flavor { VANILLA, CHOCOLATE; }  
...  
switch (icecream.flavor()) {  
    case Flavor.VANILLA : return "vanille";  
    case Flavor.CHOCOLATE : return "chocolat";  
}  
switch (text) {  
    case "foo" :  
    case "bar" : break;  
    default :  
        System.out.println(text);  
}
```

Le sélecteur instructionnel sur les interfaces scellées

Exemple de switch avec interface scellée

```
sealed interface Shape permits Square, Circle { ... }  
...  
public double area(Shape shape) {  
    switch (shape) {  
        case Square sq :  
            double side = sq.side();  
            return side * side;  
        case Circle circ :  
            double radius = circ.radius();  
            return Math.PI * radius * radius;  
    }  
}
```

- Le `switch` permet de distinguer une référence selon chaque implémentation de l'interface scellée.
- La référence testée est donnée par l'expression sélecteur,
- Dans chaque cas (exemple : `case Square sq`) :
 - on déclare une nouvelle variable (`sq`),
 - du type d'une des implémentations (`Square`),
 - contenant la référence testée (`sq == shape`),
 - on donne des instructions pour gérer ce cas.

Test d'exhaustivité

Erreur de compilation si on n'a pas pris en compte tous les cas possibles (ou utilisé un cas `default`).

```
sealed interface Shape permits Square, Circle, Triangle
{ ... }
...
public double area(Shape shape) {
    switch (shape) {
        case Square sq :
            double side = sq.side();
            return side * side;
        case Circle circ :
            double radius = circ.radius();
            return Math.PI * radius * radius;
    } // Erreur : manque le cas Triangle
}
```

Le sélecteur expressionnel

Deux formes de switch

On a vu le `switch` comme structure de contrôle.
Il existe aussi une forme `expressionnelle` :

```
sealed interface Shape permits Square, Circle { ... }  
...  
public double area(Shape shape) {  
    return switch (shape) {  
        case Square sq ->  
            sq.side() * sq.side();  
        case Circle circ ->  
            Math.PI * circ.radius() * circ.radius();  
    };  
}
```

Remarques

- Différence syntaxique : `->` plutôt que :
- En terme droit de `->`, on donne une `expression`, et non pas `des instructions`.
- Pas besoin de `break`.
- Possible d'utiliser `default` (à éviter néanmoins).

Le mot réservé `yield`

Que faire si dans un cas, l'expression résultat demande plusieurs instructions pour se calculer ?

Solution 1 : définir une méthode et y faire appel.

Solution 2 : utiliser `yield` dans un bloc d'instructions :

```
double area =  
    switch (shape) {  
    case Square sq -> sq.side() * sq.side();  
    case Circle circ -> {  
        double radius = circ.radius;  
        yield Math.PI * radius * radius;  
    };  
};
```

similaire à un `return` limité au sélecteur.

Interfaces scellées

On souhaite représenter des billets de train :

- les billets standards indiquent simplement le trajet (gare de départ et destination),
- les billets avec réservation indiquent en plus une date et un horaire de départ, ainsi qu'un numéro de place,
- les billets illimités permettent de se déplacer sur tout le réseau ferroviaire entre deux dates.

Solution

Utiliser une **interface scellée**, permettant de représenter une alternative entre plusieurs possibilités :

- une classe pour chaque possibilité : standard, avec réservation, illimité,
- une interface scellée englobant les trois possibilités sous un seul type.

Différence avec une interface standard : on interdit tout autre possibilité (**scellée**).

Exemple : consulter `cours4.trainTicket.TrainTicket`.

Contraintes sur les classes permises.

Les options permises peuvent être des classes ou des interfaces, et **doivent être définies**.

On peut utiliser :

- des enregistrements ;
- des classes non-extensibles, déclarées avec **final** ;
- d'autres interfaces ou classes scellées, déclarées avec **sealed** ;
- d'autres interfaces ou classes non-scellées, déclarées avec **non-sealed**. Ces dernières peuvent alors être étendues sans contrainte.

Utilisation du switch

La structure de contrôle `switch` permet de retrouver parmi les classes permises, laquelle correspond à une instance :

```
public static boolean isDeparture(TrainTicket ticket,
                                   Station station) {
    switch (ticket) {
        case StandardTicket stdTicket :
            return stdTicket.departure().equals(station);
        case BookingTicket bkgTicket :
            return bkgTicket.departure().equals(station);
        case UnlimitedTicket ultdTicket :
            return true; // any station is a possible departure
    }
}
```

Conclusion

On peut former des données complexes :

- par agrégation de données plus simples (classes standards, enregistrement),
- par alternative entre données plus simple (énumération, interfaces scellées).

On accède à ces données :

- par des accesseurs dans les agrégations,
- par le `switch` dans les alternatives.

C'est le fait de sceller l'interface qui permet l'usage du `switch`, le nombre de cas étant alors déterminé.

Énumérations

Énumération

Les **énumérations** sont des classes ayant un nombre **fini** et **prédéterminé** d'instances (connu au moment de programmer), permettant de représenter des données, comme par exemple :

- les jours de la semaine, les mois de l'année,
- les couleurs d'un jeu de cartes,
- l'état d'un système (actif, dormant, en attente).

Exemple de syntaxe

On définit une énumération dans un fichier dédié (comme pour une classe normale) :

```
public enum Suit {  
    SPADES, HEARTS, DIAMONDS, CLUBS;  
}
```

- Il n'y a que 4 objets de la classe Suit : Suit.**SPADES**, Suit.**HEARTS**, ...
- on peut tester l'égalité des Suit avec ==,
- on peut utiliser la méthode statique Suit.**values()** pour itérer sur toutes les options dans une boucle **for**.

```
for (Suit suit : Suit.values()) {  
    sout.println("Ace of " + suit.toString());  
}
```

La structure de contrôle `switch` accepte les `enum` :

```
public boolean isRed(Suit suit) {  
    switch (suit) {  
        case Suit.HEARTS :  
        case Suit.DIAMONDS : return true;  
        default : return false;  
    }  
}
```

Les options de l'`enum` sont considérées comme des littéraux.

Méthodes par défaut

- Les `enum` ne sont pas instanciables ! Les instances existent et sont utilisables immédiatement (`Suit.HEART`, ...),
- les instances possèdent des méthodes (par exemple, `compareTo`, `toString`, ...) qui peuvent être redéfinies (`@Override`).

```
public enum Suit {  
    SPADES, HEARTS, DIAMONDS, CLUBS;  
  
    @Override  
    public String toString() {  
        switch (this) {  
            case SPADES : return "spades";  
            case HEARTS : return "hearts";  
            case DIAMONDS : return "diamonds";  
            case CLUBS : default : return "clubs";  
        }  
    }  
}
```

Méthodes et constructeurs

- Méthodes et propriétés (`final` fortement conseillé) possibles ;
- constructeur `privé` possible, mais pas `public` ;
- les arguments du constructeur sont donnés à la déclaration de chaque option (cf. `cours4.Suit`) ;
- les `enum` peuvent implémenter des interfaces.

Consulter les exemples :

- `cours4.Month` ;
- `cours4.Suit`.